

HabiTrack

(One Place for All Dwelling Needs)

*A Project Report submitted in partial fulfilment of the requirements
for the award of the degree of*

Master of Computer Applications

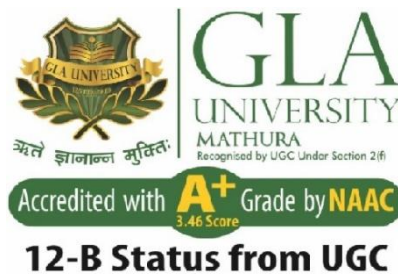
By

Khushboo (2484200097)
Himanshu Samadhiya (2484200078)
Vishnu Kumar (2484200218)

Group No. 09

Under the Guidance of
Ms. Deepti Gautam

Department of Computer Engineering & Applications
Institute of Engineering & Technology



GLA University
Mathura- 281406, INDIA
April 2026

Declaration

I hereby declare that the work which is being presented in the MCA Project “**HabiTrack (One Place for All Dwelling Needs)**”, in partial fulfilment of the requirements for the award of the *Master of Computer Applications* and submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is an authentic record of my own work carried under the supervision of **Ms. Deepti Gautam Assistant Professor Dept. of CEA.**

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign _____

Khushboo
University Roll No.: 2484200097

Sign _____

Himanshu Samadhiya
University Roll No.: 2484200078

Sign _____

Vishnu Kumar
University Roll No.: 2484200218

Certificate

This is to certify that the above statements made by the candidate are correct to the best of my/our knowledge and belief.

Supervisor
(Ms. Deepti Gautam)
Assistant Professor
Dept. of Computer Engg, & App.

Project Co-Ordinator
(Dr. Sachendra Singh Chauhan)
Assistant Professor
Dept. of Computer Engg, & App.

Program Co-Ordinator
(Mr. Siddharth Singh)
Assistant Professor
Dept. of Computer Engg, & App.

Date:

Acknowledgement

We would like to express our sincere gratitude to **Ms. Deepti Gautam Assistant Professor Dept. of CEA.** for his valuable guidance, continuous encouragement, and support throughout the development of our project titled “HabiTrack (One Place for All Dwelling needs)”. His insights and feedback have greatly contributed to the successful completion of this work.

We also thank the head of the project **Dr. Sachendra Singh Chauhan** and staff of the Institute of Engineering & Technology, GLA University Uttar Pradesh, Mathura, for providing us with the necessary academic environment, resources, and assistance during the preparation of this project.

We are deeply grateful to our families, friends, and classmates for their moral support, cooperation, and motivation at every stage of this work.

Finally, we extend our thanks to everyone who contributed directly or indirectly to the successful completion of this project.

Sign _____

Khushboo

University Roll No.: 2484200097

Sign _____

Himanshu Samadhiya

University Roll No.: 2484200078

Sign _____

Vishnu Kumar

University Roll No.: 2484200218

Abstract

HabiTrack: One Place for All Dwelling Needs is an innovative full-stack property portal designed to transform real estate experience by offering a unified platform for property discovery, management, and interaction. Built using modern technological solutions, the system aims to simplify searching, viewing, and managing real estate listings through an intuitive and interactive user interface. Its goal is to eliminate the complexities associated with traditional property portals by delivering a smooth, user-centric experience.

Developed using the MERN stack (MongoDB, Express.js, React, Node.js), the platform integrates robust backend services with responsive UI/UX design principles. Features such as map-based property search, real-time messaging, and personalized user dashboards enhance the discovery process and improve overall user engagement. The system also supports secure authentication, role-based access, and property posting capabilities that assist buyers, sellers, and agents in managing property data efficiently. A key technical highlight of HabiTrack is its scalable architecture capable of handling large volumes of dynamic property information while maintaining optimal performance across devices. By implementing modern security protocols and responsive design techniques, the platform ensures reliable data protection and seamless usage on phones, tablets, and desktops. Its intuitive interface, combined with powerful filtering and communication tools, supports a smooth transition from property search to final transaction.

Evaluation results demonstrate that HabiTrack delivers a feature-rich, efficient, and transparent real estate experience, outperforming traditional property portals in terms of search accuracy, user satisfaction, and interaction flow. The project not only showcases the effectiveness of modern web technologies in solving real-world challenges but also serves as a foundation for future advancements in digital property management systems, scalable full-stack architectures, and interactive web applications.

CONTENTS

Declaration	ii
Certificate	ii
Acknowledgement	iii
Abstract	iv
List of Tables	v
List of Figures	vi
Chapter 1: Introduction	1-5
1.1 Overview and Motivation	2
1.2 Objectives	3
1.3 Summary of Similar Systems	4
1.4 Report Organization	5
Chapter 2: Software Requirement Analysis	6-18
2.1 Requirement Analysis	6
2.1.1 Functional Requirements	6
2.1.2 Non-Functional Requirements	7
2.2 Feasibility Analysis	8
2.2.1 Technical Feasibility	8
2.2.2 Economic Feasibility	8
2.2.3 Operational Feasibility	9
2.2.4 Time Feasibility	9
2.3 Module Description	9
2.3.1 Buyer/Renter Module	10
2.3.2 Property Owner Module	10
2.3.3 Property Agent Module	11
2.4 Use Case Description	12
2.4.1 Buyer/Renter Use Case	12
2.4.2 Property Owner Use Case	14
2.4.3 Property Agent Use Case	17
2.5 System Architecture	18

Chapter 3: System Design	20-26
3.1 Data Flow Diagrams	20
3.1.1 Level 0 DFD	20
3.1.2 Level 1 DFD	21
3.1.3 Level 2 DFD	22
3.2 UML Diagrams	23
3.2.1 Class Diagram	23
3.2.2 Sequence Diagram	24
3.3 ER Diagram	26
Chapter 4: Implementation	29-39
4.1 Technology Used	29
4.1.1 Frontend Technologies	29
4.1.2 Backend Technologies	30
4.2 System Modules	30
4.2.1 Buyer/Renter Module	31
4.2.2 Property Owner / Agent Module	31
4.2.3 Admin Module	32
4.3 Backend Implementation	32
4.3.1 REST API Development	33
4.3.2 Database Implementation	34
4.3.3 Authentication & Security	35
4.3.4 Business Logic	36
4.4 Frontend Implementation	37
4.4.1 User Interface Development	37
4.4.2 Component Based Design	37
4.4.3 Communication with Backend	38
4.4.4 Displaying Analytics	38
4.5 Integration	38
4.5.1 API Integration	38
4.5.2 Module Integration	39
4.5.3 Testing & Debugging	39

Chapter 5: Testing & Results	40-44
5.1 Testing Methods	40
5.1.1 Unit Testing	40
5.1.2 Integration Testing	40
5.1.3 Functional Testing	41
5.1.4 User Interface (UI) Testing	41
5.1.5 Performance Testing	41
5.1.6 Security Testing	42
5.2 Test Cases	42
5.3 Result & Screenshots	44
Chapter 6: Conclusion	49
Chapter 7: Summary	50
Appendices	51
References	53

List of Tables

1. Table 5.2.1 – Login Test Case	42
2. Table 5.2.2 – Invalid Login Test Case	42
3. Table 5.2.3 – Property Search Test Case	43
4. Table 5.2.4 – Property Creation Test Case	43
5. Table 5.2.5 – Admin Adds User Test Case	43
6. Table 5.2.6 – Buy/Rent Property Test Case	44

List of Figures

1. Figure 2.4.1 – Use Case Diagram of User	14
2. Figure 2.4.2 – Use Case Diagram of Property Owner	16
3. Figure 2.4.3 – Use Case Diagram of Property Agent	18
4. Figure 2.5 – Sequence Diagram for Sign In	19
5. Figure 3.1.1 – Level 0 Data Flow Diagram	20
6. Figure 3.1.2 – Level 1 Data Flow Diagram	21
7. Figure 3.1.3 – Level 2 Data Flow Diagram	22
8. Figure 3.2.1 – Class Diagram	24
9. Figure 3.2.2 – Sequence Diagram	26
10. Figure 3.3 – ER Diagram	28
13. Figure 5.3.1– Login Page	44
14. Figure 5.3.2 – Home Page	45
15. Figure 5.3.3 – Property Listing Page	45
16. Figure 5.3.4 – Search Property Listing Page	45
17. Figure 5.3.5 – Contact Page	46
18. Figure 5.3.6 – Owner Profile & Chat Page	46
19. Figure 5.3.7– Property Buy/rent Payment Page	47
20. Figure 5.3.8 – Admin Dashboard	47
21. Figure 5.3.9 – User in Admin Dashboard	48
22. Figure 5.3.10 - Payment Interface	48

Chapter 1

Introduction

Today's real estate experience has become increasingly challenging for both property seekers and property owners. Users often face scattered and unreliable information across multiple platforms, making it difficult to compare properties, evaluate features, or make informed decisions. Many individuals report that existing portals provide **“insufficient details, inconsistent listings, and unclear communication channels,”** which complicates the entire property-search journey. As a result, users frequently experience confusion, delays, and frustration while trying to find a suitable home.

On the other side, property owners and agents struggle with the lack of a centralized system to manage their listings. Without modern digital tools, many owners face **“limited control over property updates and minimal visibility to potential buyers or tenants,”** reducing their ability to reach the right audience. This leads to missed opportunities and inefficient property management practices.

Furthermore, most available platforms focus only on basic listing features and fail to offer a smooth end-to-end experience. Users often navigate multiple websites for details such as property images, amenities, pricing, location insights, and agent communication. This fragmented process results in **inconsistent access to property information, limited personalized recommendations, and overall inefficiency** in real estate decision-making.

These challenges – from scattered property data to limited management tools – highlight the need for a unified, technology-driven solution. **HabiTrack: One Place for All Dwelling Needs** is designed to address these issues by providing a modern, integrated, and user-friendly platform that simplifies property browsing, interaction, and management for all users.

1.1 Overview and Motivation

To address the growing challenges in the real-estate search and management experience, we propose **HabiTrack – (One Place for All Dwelling Needs)** designed to simplify how users discover, evaluate, and manage properties. The motivation behind HabiTrack is to bring scattered real-estate information into one centralized, structured, and interactive platform. In the current digital landscape, users often depend on multiple listing sites, agents, and social platforms, leading to inconsistent property data, unclear communication, and limited decision-making support.

Recent advancements in web technologies and digital property management show that integrated platforms can significantly enhance user engagement and transparency. Modern real-estate systems are becoming “**data-rich, personalized, and highly interactive, helping users make better purchase and rental decisions.**” By applying these principles, HabiTrack aims to deliver a seamless property-search experience supported by responsive interfaces, reliable data, and efficient user interaction.

Moreover, many property seekers and owners previously struggled with a fragmented real-estate process—browsing properties on one platform, communicating on another, and relying on different sources for price comparisons or neighbourhood insights. This highlights the need for a single system that brings together **property listings, detailed information, owner/agent management, and user interaction tools** in one place.

HabiTrack aligns with current trends in digital real-estate solutions, where individuals and agencies increasingly adopt online platforms to manage property transactions. By offering a unified and user-friendly environment, HabiTrack addresses these gaps and aims to improve the overall experience of property discovery, communication, and management.

1.2 Objectives

The main objectives of the **HabiTrack Property Portal** are:

- **Centralized Property Listings:** Provide a unified platform where users can browse, compare, and explore properties with complete details such as price, amenities, location, and images.
- **Property Management Tools:** Allow property owners and agents to add, update, and manage their listings efficiently through a dedicated dashboard.
- **Advanced Search and Filters:** Offer users intuitive search options—such as budget range, property type, location, and size—to streamline the property discovery process.
- **User Interaction and Communication:** Enable seamless communication between users and property owners/agents through integrated contact or inquiry features.
- **Secure Authentication:** Implement user registration, login, and role-based access control to ensure secure and authorized access to the platform.
- **Responsive and User-Friendly Interface:** Ensure that the platform works smoothly across devices with a clean, modern, and easy-to-use interface.
- **Scalable Backend Architecture:** Use Node.js and MongoDB to build a scalable system capable of handling property data, user interactions, and future feature expansion.

These features collectively aim to create a more transparent and efficient real-estate experience. Providing centralized property data and structured management tools addresses the common challenge of “**fragmented listings and inconsistent property information**” faced by users. By achieving these objectives, HabiTrack strives to bridge the gap between property seekers and property providers through a reliable, technology-driven platform.

1.3 Summary of Similar Systems

Several existing real-estate platforms like **99acres**, **MagicBricks**, **NoBroker**, and **Housing.com** offer property listings, filters, and basic search tools. These portals have evolved into broad “property discovery ecosystems,” providing features such as price trends, map views, and owner–agent contact options. Their wide reach makes them popular choices for buyers, renters, and property owners.

However, these platforms also have noticeable limitations. Much of the content remains **static**, and users often encounter outdated or incomplete listings. Searching for a suitable property requires navigating multiple pages without personalized recommendations or consolidated insights. Although the platforms provide detailed listings, they rarely adapt to individual user preferences or offer integrated support like price comparisons, locality analytics, or guided property suggestions.

Additionally, property owners and agents face challenges managing their listings. Many portals require paid boosts or subscriptions, and the backend tools for updating or tracking property performance are either limited or unintuitive. As a result, miscommunication, inefficiencies, and missed opportunities remain common. These limitations highlight the need for a more unified, transparent, and user-centric platform—one that **HabiTrack** aims to fulfill by integrating organized property management, updated listings, and a streamlined browsing experience in one place.

1.4 Report Organization

This report is structured into a series of chapters that collectively describe the design, development, and implementation of the HabiTrack property portal. Each chapter focuses on a specific aspect of the system to ensure clarity, coherence, and a logical flow of information. The intention is to give readers a comprehensive understanding of both the problem domain and the technical solutions implemented.

The early chapters introduce the motivation, objectives, and background of the project, providing the context required to understand why a unified real-estate platform like HabiTrack is necessary. Following this, the report moves into the system analysis and design components, covering software requirements, architectural models, database structures, and detailed design diagrams. These chapters highlight how the system was conceptualized and organized before development.

The later chapters focus on implementation, testing, and evaluation. They describe the development process, the technologies used, the testing strategies applied, and the results obtained. The final chapter concludes the report by summarizing the project outcomes and suggesting potential enhancements. Overall, this organization ensures that the reader can follow the project's progression from initial motivation to final implementation in a clear and systematic manner.

Chapter 2

Software Requirements Analysis

Software requirement analysis is a crucial phase that identifies what the **HabiTrack** property portal should achieve and how it should operate. It focuses on understanding user needs, analysing gaps in existing real-estate platforms, and ensuring that the final system aligns with practical property search and management processes. The gathered requirements help define system boundaries, core functionalities, user expectations, performance goals, and operational constraints.

2.1 Requirement Analysis

Requirement analysis identifies what the system must do and how it should perform. It includes:

2.1.1 Functional Requirements

1. User Registration & Login: Users can create accounts by providing their name, email, password, and selecting a role (Buyer/Renter, Property Agent, or Property Owner), and can securely log in using their credentials. Password verification, authentication, and a forgot-password workflow are fully supported.

2. User Profile Management: Users can view and update their personal information, including username, email, and avatar, and access their saved property posts under My List. Property Owners can also create new property listing posts.

3. Property Search Module: Users can search for properties by city, price range, and type, with results displayed according to their filters. Each result shows the property name, image, price, key details, quick actions (save, favourite, etc.), and can be viewed on a map.

4. Home Dashboard & Navigation: Users see the homepage introducing HabiTrack, with featured property categories and search fields. The system also provides smooth navigation to pages like Home, About Us, Contact, and Agents.

5. Property Listing Management: Users can view detailed information for each property, including images, price, utilities, pet policy, room sizes, nearby places, and additional highlights. The interface features a gallery view of multiple images and a location map.

6. Messaging & Communication: Users can view and send chat messages with agents or other users, with each message showing the sender, timestamp, and history. New messages can be sent instantly.

7. Map Integration: Properties are displayed on an interactive map with markers showing basic details when clicked. Users can zoom and pan to explore additional properties.

8. Wishlist / Save Property Feature: Users can save properties to their personal My List, where they can later view, access, or remove them.

9. User Role Support (Buyer / Renter): During registration, users select a role (Buyer/Renter, Property Agent, or Property Owner), which personalizes property suggestions and navigation flows.

10. Logout & Session Handling: Users can securely log out from their accounts, with sessions maintained during navigation and reset upon logout.

2.1.2 Non-Functional Requirements

1. **Performance:** The system should load key pages—such as home, search, and property details—quickly and respond smoothly during interactions like filtering and map navigation.
2. **Scalability:** The application must support an increasing number of users without slowing down, using a backend and database that can scale efficiently.
3. **Security:** User data must be protected through encrypted passwords, secure login sessions, and proper access control to prevent unauthorized usage.
4. **Usability:** The interface should be simple, intuitive, and consistent across screens, with full responsiveness on desktop and mobile devices.

5. **Accuracy:** Search results, filter outputs, and map locations must correctly reflect stored data so users always see reliable property information.
6. **Reliability:** The system should remain stable with minimal downtime and handle errors gracefully without interrupting user activity.
7. **Maintainability:** The codebase should be clean and easy to update, and admins should be able to manage property listings without modifying the code.
8. **Compatibility:** The website should function properly across all major browsers and device types with consistent design and performance.

These requirements ensure a reliable, secure, and user-friendly real-estate platform that can scale smoothly as user demand and property data grow.

2.2 Feasibility Analysis

The feasibility study evaluates whether the real-estate platform is practical, achievable, and sustainable.

2.2.1 Technical Feasibility

The technologies required for this project—modern web frontend frameworks, server-side JavaScript runtimes, and cloud-supported databases—are widely available, reliable, and open-source. Standard web technologies such as **HTML**, **CSS**, **JavaScript**, and frameworks like **React** can be used to develop the user interface. **Node.js** can manage server-side logic, while databases like **MongoDB** can store property listings, user details, and search data. Libraries and APIs are also available for essential features such as **maps integration**, **user authentication**, **image storage**, and **real-time messaging**. The expected user load and data volume of a property search system can be efficiently handled by standard cloud infrastructure, making the project technically feasible.

2.2.2 Economic Feasibility

Because the system relies on free and open-source tools, development costs remain low. Hosting costs are minimal (simple server or cloud). No special hardware is needed—only a basic server and standard user devices such as laptops or mobile

phones. This makes the system economically practical for small startups, academic projects, or organizations with limited budgets.

2.2.3 Operational Feasibility

The platform aligns well with the needs of buyers, renters, and property managers. Users only require a web browser to search for properties, save posts, or view details. The admin interface allows easy management of property listings, property details, and images without any technical expertise. Training requirements are minimal due to the simple and intuitive design. The system effectively addresses common operational challenges in property search, such as scattered information, lack of filtering options, and difficulty in comparing listings.

2.2.4 Time Feasibility

The planned project timeline is realistic. Core functionalities—user registration, property posting, search filters, map integration, and profile management—can be developed incrementally. Existing libraries and frameworks help accelerate development and reduce coding time. Overall project tasks such as design, development, testing, and deployment can be completed within a typical academic or project timeframe.

Overall, the **HabiTrack** (One Place for All Dwelling needs) is fully feasible in terms of technology, cost, operations, and timeline.

2.3 Module Description

The system is divided into three main functional modules: **Buyer/Renter Module**, **Property Owner Module**, and **Property Agent Module**. Each module groups related features and workflows, ensuring smooth navigation, easier development, and efficient management of the real-estate platform.

2.3.1 Buyer/Renter Module

The Buyer/Renter Module is designed for users who want to search, filter, and shortlist properties for purchase or rental. It provides the following functionalities:

1. **Registration/Login:** Users create an account by entering details such as full name, email, and selecting their role (Buyer/Renter). The system verifies credentials and redirects them to their personalized dashboard.
2. **Dashboard:** After login, users view recommended properties, recent searches, and saved listings. The dashboard offers quick access to filters, property alerts, and location-based suggestions.
3. **Property Search & Filters:** Buyers/Renters can browse properties based on price range, location, room count, furnishing status, and property type. The system fetches relevant listings using fast and optimized search.
4. **Property Details & Comparison:** Users can open detailed property pages showing images, amenities, owner/agent details, and map location. They can also compare multiple properties side-by-side.
5. **Shortlist & Save Properties:** Users can save favourite properties to their shortlist for later viewing. The system stores this data in their profile.
6. **Contact Owner/Agent:** Users may contact property owners or agents using in-app messaging or request a callback/visit.
7. **Profile Management:** Buyers/Renters can update their personal information, reset their password, and manage notification preferences.

This module focuses on helping users explore properties easily while keeping navigation simple and intuitive.

2.3.2 Property Owner Module

The Property Owner Module is for individuals who want to list their properties for sale or rent. It includes the following functionalities:

1. **Owner Login:** Owners authenticate using their registered email and password. Role-based controls ensure they access only relevant tools.

2. **Property Listing Management:** Owners can create new listings by adding property details such as price, address, photos, amenities, furnishing type, and availability status. They can edit or remove listings at any time.
3. **Inquiry Management:** Owners receive inquiries directly from Buyers/Renters. They can view messages, reply, schedule visits, and track user interest.
4. **Property Status Updates:** Owners can update listing availability (Available, Sold, Rented), price changes, and visibility settings.
5. **Analytics & Insights:** Owners can view the total number of views, inquiries received, and user engagement patterns for each property.
6. **Profile & KYC Verification:** Owners can update their details and optionally complete KYC for verified listings, increasing trust and visibility.
7. **Hire Property Agents:** Property owners have the option to hire professional property agents to manage the selling or renting process more efficiently.

By centralizing listing control, this module helps property owners efficiently manage their properties and interact with potential buyers or tenants.

2.3.3 Property Agent Module

The Property Agent Module is built for real-estate agents who act as intermediaries between property owners and buyers/renters. It offers advanced tools for managing multiple properties and customers.

1. **Agent Authentication:** Agents log in with dedicated credentials. Additional verification may be applied for professional authentication.
2. **Manage Multiple Listings:** Agents can organize multiple property listings for various owners. They can categorize properties by type, location, or client.
3. **Lead & Client Management:** Agents can track inquiries from both buyers and renters, respond to queries, schedule visits, and maintain communication logs.
4. **Owner & Buyer Coordination:** Agents manage interactions between property owners and potential customers, ensuring smooth communication and faster property transactions.
5. **Advanced Analytics:**

This includes:

- Property-wise user engagement
- Lead conversion rates
- Trending locations and price ranges
- Performance summaries for all managed properties

6. **Reputation & Review Management:** Agents can view feedback from users, helping them improve service quality and ranking within the platform.

The Agent Module ensures efficient multi-property handling, streamlined communication, and professional management of real-estate operations.

2.4 Use Case Descriptions

HabiTrack (One Place for All Dwelling Needs) supports three primary actors: **Buyer/Renter**, **Property Owner**, and **Property Agent**. Each actor interacts with the system through specific use cases such as account creation, property browsing, posting listings, and communication.

The following section provides detailed descriptions for each use case.

2.4.1 Buyer/Renter Use Cases

Use Case 1: Sign Up / Sign In

- **Actor:** Buyer/Renter
- **Description:** The user creates an account by entering full name, email, password, and selecting the role *Buyer/Renter*. An existing user can sign in using their credentials.
- **Outcome:** The user is authenticated and redirected to the property browsing dashboard.

Use Case 2: Search Properties

- **Actor:** Buyer/Renter
- **Description:** The user searches for properties by entering city, location, price range, or category.

- **Outcome:** The system displays a list of matching properties with essential details and images.

Use Case 3: View Property Details

- **Actor:** Buyer/Renter
- **Description:** The user selects a specific property to view full details such as location, price, amenities, room sizes, gallery images, utilities, and nearby places.
- **Outcome:** The user gets complete information needed to evaluate the property.

Use Case 4: Save or Shortlist Property

- **Actor:** Buyer/Renter
- **Description:** The user can save/shortlist properties to revisit later from their profile.
- **Outcome:** The property is added to the user's personal "My List".

Use Case 5: Contact Property Owner/Agent

- **Actor:** Buyer/Renter
- **Description:** The user chats or sends inquiries directly to the property owner/agent.
- **Outcome:** Communication is established to schedule visits or negotiate details.

Use Case 6: Manage Profile

- **Actor:** Buyer/Renter
- **Description:** The user can edit personal details such as name, profile photo, or contact information.
- **Outcome:** Updated profile is stored successfully.



Figure 2.4.1 Use Case Diagram of User

2.4.2 Property Owner Use Cases

Use Case 1: Sign Up / Sign In

- **Actor:** Property Owner
- **Description:** The owner creates an account by entering basic details or logs in using existing credentials.
- **Outcome:** The owner is authenticated and redirected to the property management dashboard.

Use Case 2: Add / Upload Property

- **Actor:** Property Owner
- **Description:** The owner adds a new property by entering its details such as price, location, amenities, room descriptions, and uploading images.
- **Outcome:** The property is successfully listed on the HabiTrack platform.

Use Case 3: Manage Property Listings

- **Actor:** Property Owner
- **Description:** Allows the owner to view all submitted properties, edit information, update prices, or remove inactive listings.
- **Outcome:** Property listings remain updated and accurate for potential buyers or renters.

Use Case 4: Update Property Details

- **Actor:** Property Owner
- **Description:** The owner updates specific property elements such as images, amenities, description, or availability status.
- **Outcome:** The system saves and reflects the updated information instantly.

Use Case 5: View Interested Buyers/Renters

- **Actor:** Property Owner
- **Description:** The system shows inquiries sent by users interested in the listed property.
- **Outcome:** Owners can identify potential clients for scheduling visits or negotiations.

Use Case 6: Respond to Inquiries

- **Actor:** Property Owner
- **Description:** The owner replies to messages or queries sent by interested users regarding the property details.
- **Outcome:** Communication is established between owner and customer, aiding decision-making.

Use Case 7: Manage Profile

- **Actor:** Property Owner
- **Description:** The owner updates personal details such as name, contact number, email, or profile picture.
- **Outcome:** Updated profile information is stored successfully in the system.

Use Case 8: Hire Property Agent

- **Actor:** Property Owner
- **Description:** The owner browses available property agents, reviews their profiles, and hires an agent to help sell or rent the property faster. The owner can communicate with the agent and track progress.

- **Outcome:** A professional agent is hired to manage inquiries, conduct visits, negotiate deals, and assist in closing the property transaction efficiently.

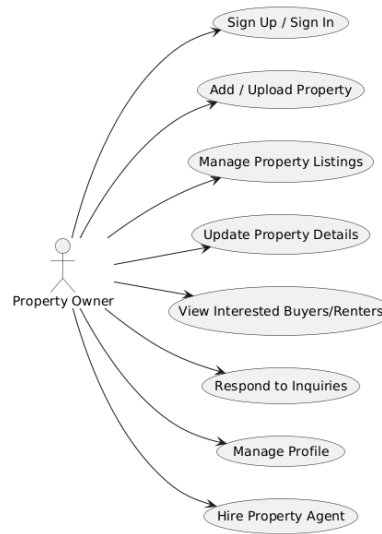


Figure 2.4.2 Use Case Diagram of Property Owner

2.4.3 Property Agent Use Cases

Use Case 1: Sign Up / Login

- **Actor:** Property Agent
- **Description:** The agent creates an account by providing necessary credentials or logs in using an existing account.
- **Outcome:** The agent is authenticated and redirected to the agent dashboard.

Use Case 2: Manage Assigned Properties

- **Actor:** Property Agent
- **Description:** The agent manages properties on behalf of owners or manages assigned listings by updating descriptions, prices, and status (available/sold/rented).
- **Outcome:** Property information remains accurate and visible to potential customers.

Use Case 3: View Customer Inquiries

- **Actor:** Property Agent
- **Description:** The system displays messages and inquiries sent by buyers/renters who are interested in listed properties.
- **Outcome:** The agent receives real-time updates about customer interest.

Use Case 4: Respond to Customer Queries

- **Actor:** Property Agent
- **Description:** The agent communicates with interested users to answer queries, provide property details, or schedule visits.
- **Outcome:** Smooth communication is established between agent and client.

Use Case 5: Schedule Property Visits

- **Actor:** Property Agent
- **Description:** The agent confirms and schedules property visits for customers, coordinating with the property owner if required.
- **Outcome:** Customers receive confirmed visit timings and instructions.

Use Case 6: Track Listing Performance

- **Actor:** Property Agent
- **Description:** The agent views details like number of views, user interest, inquiries received, and engagement metrics for each property.
- **Outcome:** The agent gains insights into property demand and can optimize listings.

Use Case 7: Manage Agent Profile

- **Actor:** Property Agent
- **Description:** The agent updates personal details such as name, contact information, email, and profile picture.
- **Outcome:** Updated profile information is saved successfully.

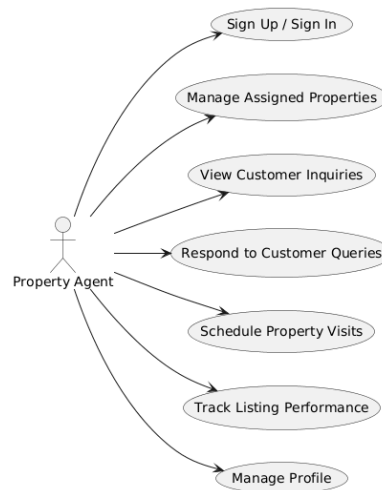


Figure 2.4.3 Use Case Diagram of Property Agent

2.5 System Architecture

The system follows a **three-tier architecture**, separating the user interface, application logic, and data storage. The key components are:

- **Frontend (Presentation Layer):** The presentation layer provides web-based interfaces for Buyers/Renters, Property Owners, and Property Agents. Built using **React.js**, along with HTML/CSS/JavaScript, it offers responsive screens for property browsing, user dashboards, property posting, bookings, and communication features. This layer handles all input/output operations—forms, filters, buttons, and interactions—and sends requests to the backend through RESTful APIs.
- **Backend (Application Logic Layer):** The backend is implemented using **Node.js and Express.js**, and it contains the core business logic of HabiTrack. It manages user authentication (via JWT), role-based access (Buyer/Owner/Agent), property management workflows, scheduling, notifications, and query handling. Controllers in this layer process incoming requests from the frontend, validate data, apply business rules, and coordinate operations with the database.
- **Database (Data Management Layer):** A NoSQL database such as **MongoDB** is used to store all persistent data required by the **HabiTrack** system. This includes user accounts, property listings, images, visit requests, customer inquiries, and notifications. The database schema is designed to support the ER model described in Chapter 3,

ensuring clear relationships between users, properties, and interactions. Queries are used to efficiently retrieve, insert, and update data across collections, enabling real-time search, booking, and communication features.

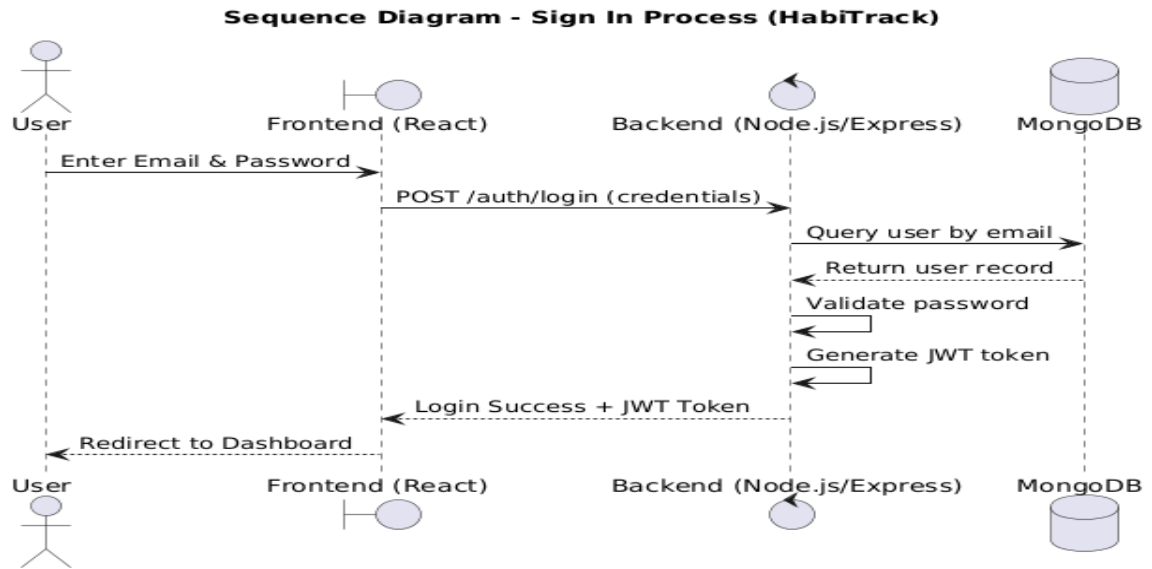


Figure 2.5 Sequence Diagram for Sign In

Chapter 3

System Design

The System Design translates the requirements into a concrete architecture and models. This chapter details the system's architecture, data flow diagrams, UML diagrams, and the ER diagram of the database.

3.1 Data Flow Diagrams

Data Flow Diagrams (DFDs) illustrate how data moves through the system. We provide diagrams at multiple levels of detail.

3.1.1 Level 0 (Context Diagram):

The Level-0 DFD represents the **HabiTrack** system as a single unified process interacting with external entities. The primary external users are the **Buyer/Renter**, **Property Owner**, and **Property Agent**. Buyers/Renters send requests such as searching for properties, viewing details, saving listings, and sending messages. In response, the system provides search results, property information, and communication updates.

Property Owners and Agents interact with the system by submitting property listings, updating details, and responding to messages. The system returns confirmations, listing status updates, and inquiries from interested users.

This context diagram abstracts the internal operations and focuses only on how external users exchange data with the main **HabiTrack** system.



Figure 3.1.1 Level 0 Data Flow Diagram

3.1.2 Level 1 DFD:

The **HabiTrack** system consists of several key subprocesses that support property browsing and management. The main processes are **User Authentication**, **Property Management**, **Search & Filtering**, and **User Communication**.

During sign-up or login, the **Authentication** process verifies user information with stored records and grants access based on the selected role (Buyer/Renter, Property Owner, or Agent). Once authenticated, owners and agents can add or manage property listings through the **Property Management** process, which stores all property details in the database.

Buyers and renters interact mainly with the **Search & Filtering** process, which retrieves properties from the database based on criteria such as location, price, and type. When a user selects a listing, the system displays detailed property information.

The **User Communication** process handles messaging between buyers/renters and property owners or agents. All messages and interactions are stored in the system for future reference.

Overall, the Level -1 DFD shows how user inputs flow through authentication, property operations, searching, and communication, while interacting with centralized data stores.

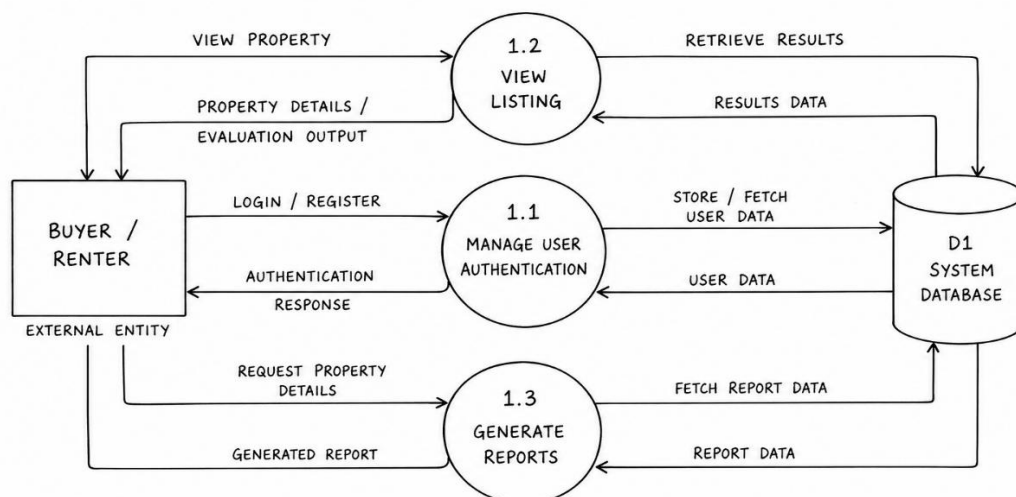


Figure 3.1.2 Level 1 Data Flow Diagram

3.1.3 Level 2 DFD:

Further decomposition details individual tasks. For instance, the User Authentication process can be split into "Verify Credentials" and "Load User Profile." The Property Management process can be split into "Validate Listing Data" and "Update Property Records." The Search & Filtering process can be split into "Process Search Criteria" and "Generate Results." The User Communication process can be split into "Route Messages" and "Store Conversation History."

These lower-level DFDs ensure every data flow is accounted for. (For brevity, we describe them conceptually; full diagrams would be included in an appendix.) Overall, the multi-level DFDs clarify the internal workings, showing exactly how user inputs traverse the HabiTrack system and what outputs result.

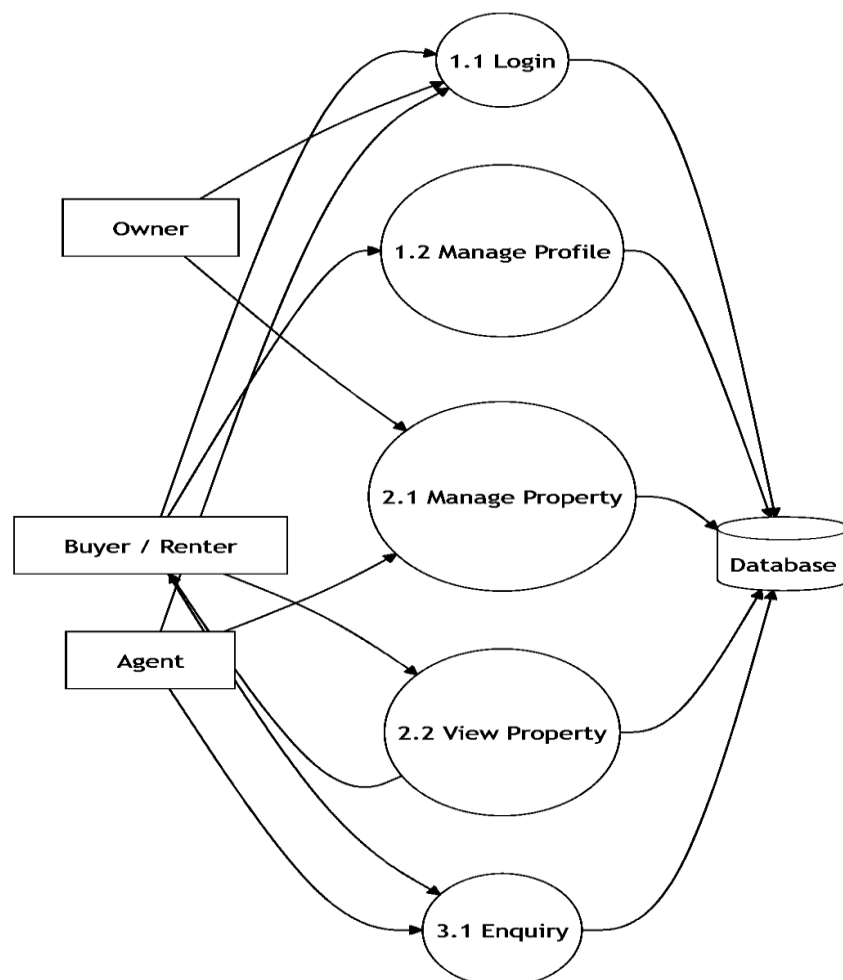


Figure 3.1.3 Level 2 Data Flow Diagram

3.2 UML Diagrams

We use UML (Unified Modelling Language) diagrams to model the system's structure and behaviour:

3.2.1 Class Diagram

The UML Class Diagram represents the core classes in the **HabiTrack** system along with their attributes, operations, and interrelationships.

Key classes include User, Property, Listing, Message, and Search.

1. **User** contains attributes such as userID, name, email, and role. It provides operations like signin(), viewProperties(), and sendMessage().
2. **Property** includes propertyID, address, price, type, and features, along with the operation updateAvailability().
3. **Listing** aggregates Property objects and maintains details such as listingID, datePosted, and status.
4. **Buyer/Renter**, a subclass of User, can search properties and save favourites using operations such as searchProperties() and saveFavorite().
5. **PropertyOwner**, a subclass of User, can create listings and manage properties using operations like createListing() and manageProperties().
6. **PropertyAgent**, a subclass of User, can manage multiple listings and communicate with clients using operations like manageListings() and contactClients().
7. The **Message** class links Users and stores messageContent, timestamp, and send/reply methods.

Associations illustrate relationships:

- A User can be either a Buyer/Renter, Property Owner, or Property Agent
- Property Owners create one or more Listings
- A Listing contains one Property

- Users exchange multiple Messages
- Buyers/Renters can save multiple favourite Properties
- Property Agents can manage properties for multiple Property Owners

This class diagram ensures the database schema and backend architecture follow the intended functional structure of the **HabiTrack** system.

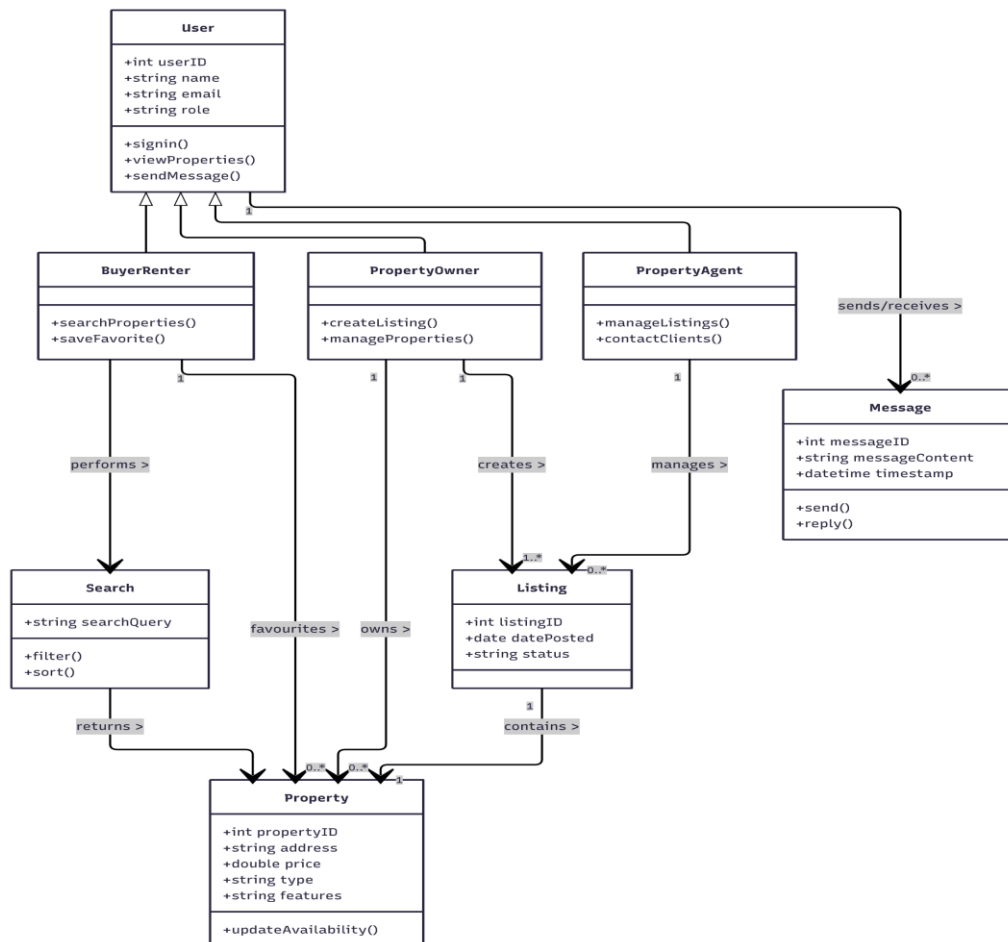


Figure 3.2.1 Class Diagram

3.2.2 Sequence Diagram

The sequence diagram illustrates the complete workflow of how a user interacts with the **HabiTrack** system while searching for and viewing properties. It shows communication between four main components: User, Frontend Application, Backend API, and the Database. The flow is as follows:

1. **User Opens Property Search Page:** The interaction begins when the User navigates to the property search page in the application. The Frontend (User App) receives this request.
2. **Frontend Requests Property Data:** The Frontend sends a GET request to the Backend API (GET /api/properties) to retrieve available property listings with initial filters.
3. **Backend Fetches Property Information:** The Backend queries the Database to fetch property metadata, images, and owner/agent details. The Database returns the complete property data to the Backend.
4. **Backend Sends Property Data to Frontend:** The Backend forwards the property dataset back to the Frontend, which then renders the property listings for the user.
5. **User Applies Advanced Filters:** The User applies additional filters such as price range, location, and property type. The Frontend sends the refined search criteria to the Backend via a POST request (POST /api/properties/filter).
6. **Backend Processes Filtered Search:** The Backend queries the Database with the advanced filters, retrieves matching properties, and prepares the filtered results.
7. **Database Returns Filtered Results:** The Database responds with the filtered property listings that match the user's criteria.
8. **Backend Returns Filtered Results to Frontend:** The Backend forwards the filtered property data to the Frontend.
9. **Frontend Displays Refined Results to User:** Finally, the Frontend presents the filtered property listings, including images, prices, and key details to the User.

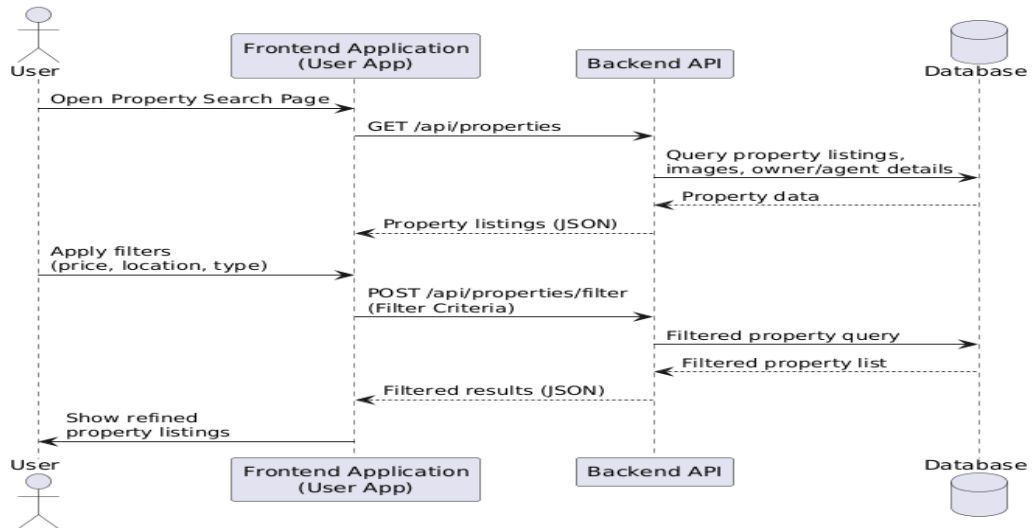


Figure 3.2.2 Sequence Diagram

3.3 ER Diagram

The Entity–Relationship (ER) diagram represents the logical data structure of the **HabiTrack** Real Estate System. It identifies the core entities, their attributes, and the relationships between them. The ER model ensures that the database design supports all system functionalities such as user management, property listing, search operations, and communication handling.

Major Entities

1. **Property Owner:** Property Owners are responsible for creating and managing property listings.
Attributes: ownerID (PK), name, email, password, phone.
2. **Buyer/Renter:** Buyers/Renters search for properties and communicate with owners.
Attributes: buyerID (PK), name, email, password, phone.
3. **Property Agent:** Property Agents assist owners with listing, promotion, and negotiation. They act as intermediaries between buyers and property owners.
Attributes: agentID (PK), name, email, phone, agencyName.
4. **Property:** Each Property is created by a Property Owner and contains multiple features.

Attributes: propertyID (PK), address, price, propertyType, bedrooms, bathrooms, squareFootage, status.

5. **Listing:** A Property has one Listing. Each Listing stores availability and display information.

Attributes: listingID (PK), propertyID (FK), ownerID (FK), agentID(FK), listingDate, expiryDate, isActive.

6. **Message:** This entity records the communication between Buyers and PropertyOwners about specific properties. Attributes: messageID (PK), senderID (FK), receiverID (FK), propertyID (FK), content, timestamp, isRead.

Relationships

- **Property Owner → Listing:**
 - A Property Owner can create many Listings.
- **Property Owner → Property**
 - A Property Owner can own many Properties.
- **Property → Listing**
 - A Property has one Listing. (One-to-one relationship)
- **Property Agent → Property**
 - A Property Agent can manage *multiple* Properties. (One-to-many)
- **Property Agent → Listing**
 - A Buyer can send *many* messages to Owners/Agents. (One-to-many)
- **Property Agent → Message**
 - A Property Agent can also receive *many* messages from Buyers. (One-to-many)
- **Buyer/Renter → Message**
 - A Buyer/Renter can send many Messages. (One-to-many)
- **Property Owner → Message**
 - A Property Owner can receive many Messages.
- **Message** acts as an associative entity between Buyer and Property Owner, representing the "Buyer contacts Owner about Property" relationship and storing communication history.

Purpose of the ER Model

This ER diagram ensures that all functional requirements of the system are covered:

- Implementing role-based user management (Owner, Agent, Buyer)
- Managing property creation, listing, and agent assignment
- Enabling buyer-owner-agent communication
- Ensuring normalized, non-redundant database structure
- Providing a solid blueprint for MongoDB/Mongoose schema design
- Maintaining consistency between backend design and real system requirements

The ER model guides the actual database design and ensures consistency between backend implementation and system requirements.

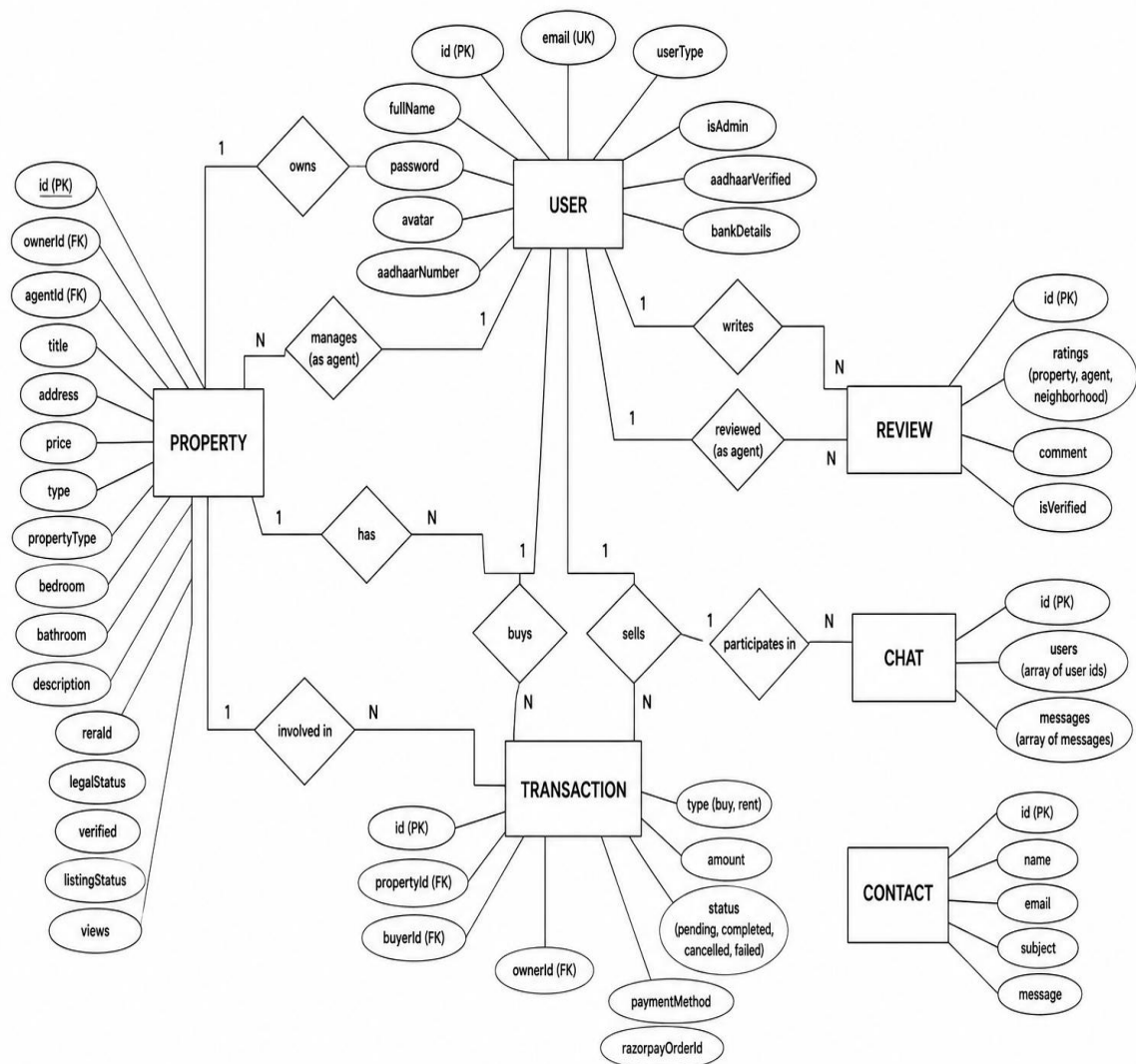


Figure 3.3 ER Diagram

Chapter 4

Implementation

Implementation is the phase where the system design is transformed into a working application. This chapter explains the technologies used, system modules, backend and frontend development processes, and how all components are integrated to form the final HabiTrack System (full-stack real estate web application).

4.1 Technologies Used

The HabiTrack System (full-stack real estate web application) is developed using a combination of modern frontend and backend technologies. These tools ensure high performance, scalability, and a smooth user experience. The technologies are organized below into two main categories: Frontend Technologies and Backend Technologies.

4.1.1 Frontend Technologies

The frontend of the system is developed using:

1. **React.js:**

Used for building a dynamic Single Page Application (SAP) with reusable components.

2. **Vite:**

Used as a fast development server and build tool.

3. **React Router DOM:**

Handles client-side routing without page reloads.

4. **React Leaflet & Leaflet:**

Used for displaying interactive maps with property locations.

5. **SCSS(SASS):**

Used for styling with variables and responsive design.

6. **React Toastify:**

Provides user notifications.

7. **Context API:**

Manages global authentication state across the application.

These technologies ensure smooth navigation, faster rendering, and a modern user experience.

4.1.2 Backend Technologies

The backend is implemented using server-side technologies to manage business logic and data processing.

- 1. Node.js:**

Provides runtime environment for executing JavaScript on the server.

- 2. Express.js:**

Framework used to build RESTful APIs.

- 3. MongoDB Atlas:**

Cloud-based NoSQL database for storing application data.

- 4. Mongoose:**

ODM library for defining schemas and interacting with MongoDB.

- 5. bcrypt:**

Used for password hashing and security.

- 6. JWT (JSON Web Token):**

Used for user authentication and authorization.

- 7. Multer:**

Handles images uploads for property listings.

- 8. dotenv & CORS:**

Manages environment variables and cross-origin requests.

These technologies provide efficient backend processing, scalability, and secure data handling.

4.2 System Module

The HabiTrack System (full-stack real estate web application) is divided into three major functional modules, each designed to handle specific roles and responsibilities. These modules ensure smooth workflow, proper access control, and efficient management of the system operations.

4.2.1 Buyer/Renter Module

The Buyer/Renter Module is designed for user who wanted to search and purchase or rent properties.

Key Features:

1. User Registration & Login:

Users can create an account and securely log in to access system features.

2. Search & Filter Properties:

Users can search properties based on location, price range, and property type.

3. View Property Details:

Users can view detailed information about properties along with map-based location.

4. Save Favourite Property:

Users can save and manage their favourite properties for future reference.

5. Buy/Rent Property:

Users can purchase or rent properties using simulated payment system.

6. Chat with Owner/Agents:

Enables direct communication with property owners or agents through the chat system.

This module ensures a smooth, interactive, and user-friendly property discovery and transaction experience

4.2.2 Property Owner / Agent Module

This Property Owner / Agent Module supports users who want to list and manage properties on the platform.

Key Features:

1. Secure Login:

Owners and agents authenticate through a secure login system using JWT-based authentication.

2. Add Property Listings:

Create new property listings with details such as price, address, type, and images.

3. Manage Listings:

Edit, update, or delete property listings at any time.

4. View Buyer Inquiries:

Access messages and inquiries sent by buyers/renters.

5. Track Property Performance:

Monitor engagement such as view and user interest.

6. Auto Geolocation:

Property location is automatically mapped using geocoding services.

Owners and agents ensure that property data remains accurate, updated, and accessible to potential buyers.

4.2.3 Admin Module

The Admin Module provides the highest level of control over the system. It ensures that overall platform management, security, and data integrity are maintained.

Key Features:

1. Manage user Accounts:

Add, edit, activate, or deactivate users (buyers, owners, agents).

2. Manage Property Listings:

View, verify, or remove property listings across the platform.

3. View System Reports:

Access analytics such as total users, properties, transactions, and revenue.

4. Monitor Transactions:

Track all buy/rent transactions performed on the platform.

5. Oversee Security & Activity Logs:

Monitor system usage, login activity, and suspicious behaviour.

This module ensures high-level administrative control, system monitoring, and secure platform operations.

4.3 Backend Implementation

The backend of the HabiTrack system is responsible for handling business logic, managing data, processing user requests, and ensuring secure communication between

the client and the server. It is developed using **Node.js and Express.js**, following a RESTful architecture, and integrates with **MongoDB Atlas** for data storage.

4.3.1 REST API Development

The backend is implemented using RESTful APIs, which serve as the communication layer between the frontend (React application) and the database.

REST APIs are designed following standard HTTP protocols and allow the system to perform operations such as data retrieval, insertion, updating, and deletion. Each API endpoint corresponds to a specific functionality of the system and returns responses in JSON format.

Key API Routes:

1. **Authentication APIs (/api/auth):**

These APIs handle user authentication and authorization processes such as registration, login, password reset, and Aadhaar OTP verification. Upon successful login, a JWT token is generated and sent to the client.

2. **Property APIs (/api/properties):**

These APIs manage all property-related operations including creating new listings, updating existing properties, retrieving property details, filtering properties based on user input, and deleting listings.

3. **User APIs (/api/user):**

These APIs handle user profile management such as updating personal details, managing saved properties, and retrieving user-specific data.

4. **Transaction APIs (/api/transactions):**

These APIs manage property buying and renting processes. They store transaction details such as buyer, owner, property, and payment information.

5. **Chat APIs (/api/chat):**

These APIs enable communication between users, allowing buyers and property owners to exchange messages in real-time-like chat threads.

6. Admin APIs (/api/admin):

These APIs are restricted to administrators and allow them to manage users, properties, and transactions, as well as monitor system performance.

All APIs follow standard HTTP methods such as GET (retrieve data), POST (create data), PUT (update data), and DELETE (remove data). This structured approach ensures modularity, scalability, and maintainability of the backend system.

4.3.2 Database Implementation

The HabiTrack System uses MongoDB Atlas, a cloud-based NoSQL database, to store and manage application data. MongoDB is chosen due to its flexibility, scalability, and ability to handle large volumes of unstructured data.

The database is structured into multiple collections, each representing a key entity in the system.

Main Collections:

1. User Collections:

Stores user-related information such as name, email, password (hashed), user role (buyer, owner, agent, admin), saved properties, and Aadhaar verification details.

2. Property Collections:

Stores property details including title, address, price, property type, number of rooms, images, geographical coordinates (latitude and longitude), and verification status.

3. Transaction Collections:

Stores records of property transactions, including buyer ID, owner ID, property ID, transaction type (buy/rent), amount, and payment details.

4. Chat Collections:

Stores communication data between users, including message threads, sender information, timestamps, and conversation history.

5. Contact Collections:

Stores user queries submitted through the contact form, including name, email, subject, and message.

The database schema is designed using **Mongoose**, which provides a structured way to define models and enforce validation rules. Relationships between collections are maintained using references, ensuring data consistency and efficient querying.

4.3.3 Authentication & Security

Security is critical aspect of the HabiTrack System. Multiple mechanisms are implemented to ensure that user's data and system operations are protected.

Security Features:

1. JWT (JSON Web Token) Authentication:

After successful login, a JWT token is generated and sent to the client. This token is used to authenticate all subsequent requests. It contains user information such as ID, role, and permissions.

2. Password Hashing (bcrypt):

User passwords are never stored in plain text. Instead, they are hashed using the bcrypt algorithm before being stored in the database, ensuring strong protection against unauthorized access.

3. Role-Based Access Control (RBAC):

The system defines different roles such as Buyer, Owner, Agent, and Admin. Access to specific resources and APIs is restricted based on these roles.

4. Protected Routes using Middleware:

Middleware functions are used to verify JWT tokens before allowing access to protected routes. Unauthorized users are denied access.

5. Aadhaar OTP Verification:

The system integrates Aadhaar-based OTP verification to ensure user identity authenticity. This adds an additional layer of trust and security.

6. Secure Data Transmission:

All communication between frontend and backend is performed using secure HTTP protocols, ensuring data integrity.

JWT tokens are stored in the browser's local Storage and included in the Authorization header of Api requests.

4.3.4 Business Logic

The business logic layer defines the system processes data and enforces rules. It acts as the core of the application, ensuring that all operations are performed correctly.

Key Functionality:

1. Property search & Filtering:

Users can search properties based on multiple criteria such as location, price range, property type, and number of rooms. The backend processes these filters and returns relevant results.

2. User Role Management:

The system ensures that users can only access features allowed for their role. For example, only owners can create property listings, and only admins can manage all users.

3. Transaction Processing:

When a user buys or rents a property, the backend validates the request, processes payment details (simulated), and stores the transaction securely.

4. Chat & Messaging Handling:

Messages between users are stored and retrieved efficiently, ensuring seamless communication

5. Property Verification & Status Updates:

Admins can verify property listings and mark them as legal or illegal, ensuring trust and reliability in the system.

This structures business logic ensures smooth operation and consistency across all modules.

4.4 Frontend Implementation

The frontend of the HabiTrack is developed using React.js, which provides a dynamic and responsive user interface. It is designed as a Single Page Application (SPA), ensuring smooth navigation and fast performance.

4.4.1 User Interface Development

The user interface focuses on providing a clean, modern, and intuitive experience for users.

Key Features:

- **Responsive Design:** The application adapts to different screen sizes, including mobile, tablet, and desktop.
- **Interactive Pages:** Includes pages such as Home, Property Listings, Property Details, User Profile, and Admin Dashboard.
- **Smooth Navigation:** React Router enables navigation without page reloads, improving user experience.
- **Visual Components:** Includes images sliders, maps, and cards for better presentation of property data.

The UI design emphasizes simplicity, usability, and accessibility.

4.4.2 Component Based Design

React follows a component-based architecture, where the UI is divided into reusable components.

Major Component Includes:

1. **Navbar:** Provides navigation across the application.
2. **Property Card:** Displays property summary information.
3. **Search Filter:** Allows users to filter properties.
4. **Map Component:** Displays property locations using Leaflet maps.
5. **Chat Component:** Enables messaging between users.

6. Dashboard Panel: Used in profile and admin section.

This modular approach improves code organization, reusability, and maintainability.

4.4.3 Communication with Backend

The frontend communicates with the backend through API calls using:

- **Fetch API or Axios** for sending HTTP requests.
- REST API endpoints for authentication, property management, transactions, and messaging.

All requests are routed through /api endpoints, responses are processed to update the UI dynamically. JWT tokens are included in request headers for secure communication.

4.4.4 Displaying Analytics

The System provides analytical insights to admin and property owners.

Analytics Features:

1. Total number of users and verified users.
2. Number of property listings and their performance.

These analytics helps stakeholder understand system performance and make informed decisions.

4.5 Integration

Integration connects the frontend, backend, and database to work as a complete system. In HabiTrack, all components interact smoothly to ensure efficient data flow and user experience.

4.5.1 API Integration

- The frontend sends requests to the backend using REST APIs.
- The backend processes requests and interact with the MongoDB database.
- Responses are returned in JSON format and displayed on the UI.

4.5.2 Module Integration

- **Authentication** connects users to the role-based dashboards (Buyer, Owner/Agent, Admin).
- **Property Module** integrate search, filters, map view, and property details.
- **Transaction Module** connects buy/rent features with user profile and records.
- **Chat Module** enables communication between users.
- **User Profile** integrate saved properties, transactions, and verifications details.

4.5.3 Testing & Debugging

- APIs tested using Postman.
- UI tested across different screen sizes.
- Errors identified and fixed at frontend, backend, and data.

Chapter 5

Testing & Results

Testing is an important phase of software development used to evaluate the functionality, performance, reliability, and security of the system. This chapter explains the testing methods used in HabiTrack and the results obtained after testing.

5.1 Testing Methods

To ensure that the HabiTrack system works correctly, multiple testing techniques were applied.

5.1.1 Unit Testing

Individual functions, APIs, and components were tested separately.

Examples:

- User Login and registration API.
- Property search and filtering logic.
- Chat and messaging function.
- Transaction processing function.
- Admin property and user management function.

This ensures that each function behaves correctly on its own.

5.1.2 Integration Testing

Integration testing was performed to verify that multiple modules work together properly.

Example:

- User login connected with role-based dashboard.
- Property search connected with property details and map view.
- Buy/rent transaction connected with user profile and transaction history.
- Chat system connected with user and property module.

- Admin actions connected with database updates.

This ensures smooth interaction between all modules and the database.

5.1.3 Functional Testing

Functional testing was performed to verify that the system meets all functional requirements.

Major Features Tested:

- User authentication.
- Property listing and search.
- Save/unsaved properties.
- Chat and messaging.
- Aadhaar OTP verification.
- Admin dashboard and analysis.

Each feature was tested from start to finish.

5.1.4 User Interface (UI) Testing

UI testing ensured that:

- Pages load correctly.
- Buttons, forms, filters, and maps work properly.
- Property cards and images display correctly.
- Layout is responsive on mobile, tablet, and desktop devices.

5.1.5 Performance Testing

Performance testing was done to evaluate how the system behaves under loads.

Example:

- Multiple users searching properties simultaneously.
- Large number of property listings loading together.
- Chat messages and notifications loading quickly.
- Dashboard analysis rendering efficiently.

5.1.6 Security Testing

Security testing ensured that the system remains safe and protected.

Tested Areas:

- Password hashing using bcrypt.
- JWT token authentication.
- Role-based routes restrictions (Buyer, Owner/Agent, Admin).
- Aadhaar OTP Verification.

5.2 Test Cases

Below is sample test cases used during testing of the HabiTrack system.

Test Case 1 – Login Test Case

Test Case ID	TC-01
Feature	User Login
Description	Verify login with valid credentials
Input	Valid emails and password
Expected Output	User should be redirected to dashboard
Actual Output	User redirected successfully
Status	Passed

Table 5.2.1 Login Test Case

Test Case 2 – Invalid Login Test Case

Test Case ID	TC-02
Module	Authentication
Test Scenario	Login with incorrect password
Precondition	User account exists
Input	Wrong password
Expected Output	Error message “Invalid Credentials”
Actual Output	Error displayed correctly
Status	Passed

Table 5.2.2 Invalid Login Test Case

Test Case 3 – Property Search Test Case

Test Case ID	TC-03
Module	Property search
Test Scenario	Search properties with filter
Precondition	Property listing must exist
Input	Location, price range, property type
Expected Output	Matching properties displayed
Actual Output	Correct property shown
Status	Passed

Table 5.2.3 Property Search Test Case

Test Case 4 – Property Creation Test Case

Test Case ID	TC-04
Module	Property management
Test Scenario	Create new property listing
Precondition	Owner/Agent authentication
Input	Property details and images
Expected Output	Property added successfully
Actual Output	Added successfully
Status	Passed

Table 5.2.4 Property Creation Test Case

Test Case 5 - Admin Adds User Test Case

Test Case ID	TC-05
Module	Admin module
Test Scenario	Add new user account
Precondition	Admin logged in
Input	User email and details
Expected Output	New User account created
Actual Output	Working fine
Status	Passed

Table 5.2.5 Admin Adds User Test Case

Test case 6 – Buy/Rent Property Test Case

Test Case ID	TC-06
Module	Transaction Module
Test Scenario	Buy or rent a property
Precondition	User logged in and property exist
Input	Payment details
Expected Output	Transaction complete successfully
Actual Output	Transaction saved successfully
Status	Passed

Table 5.2.6 Buy/Rent Property Test Case

5.3 Result & Screenshot

Testing showed that all major features of the HabiTrack system worked properly. Login, registration, Aadhaar verification, property search, transactions, and chat features functioned correctly. Property listings and admin operations also worked successfully. The interface was responsive across different devices, and the system performed efficiently and reliably.

1. Login Page

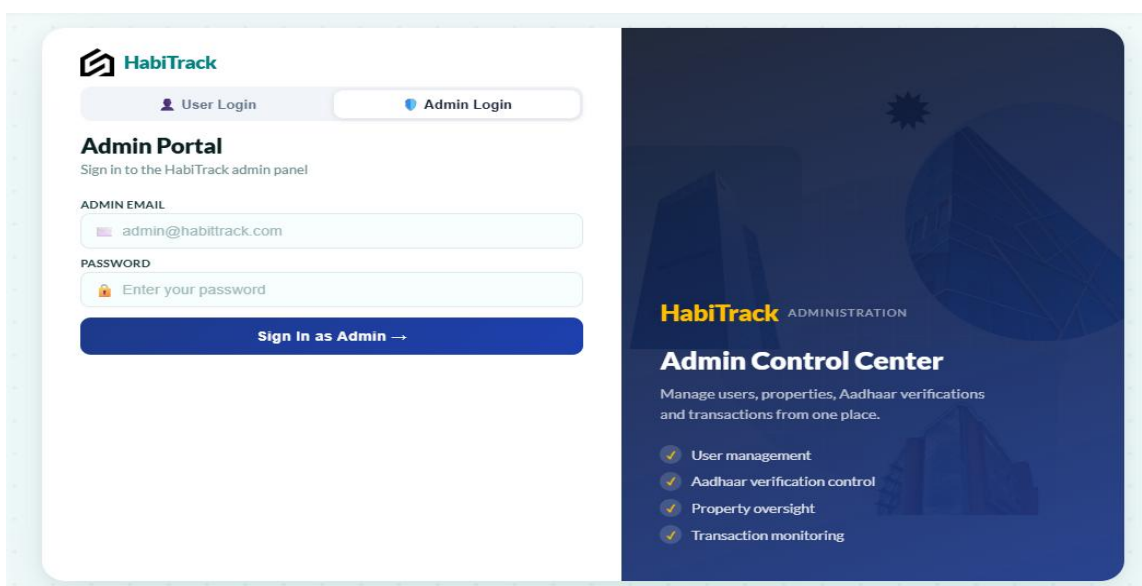


Figure 5.3.1 Login Page

2. Home Page

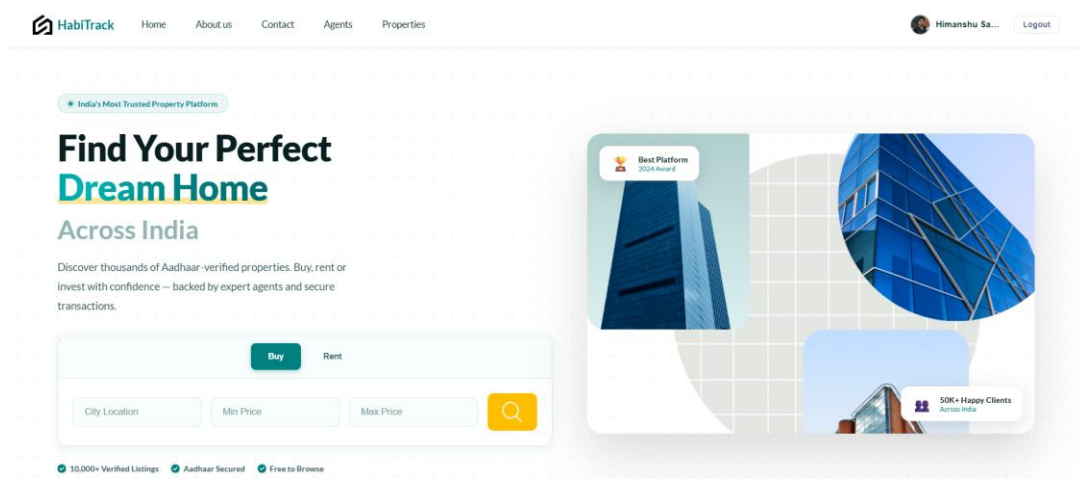


Figure 5.3.2 Home Page

3. Property Listing Page

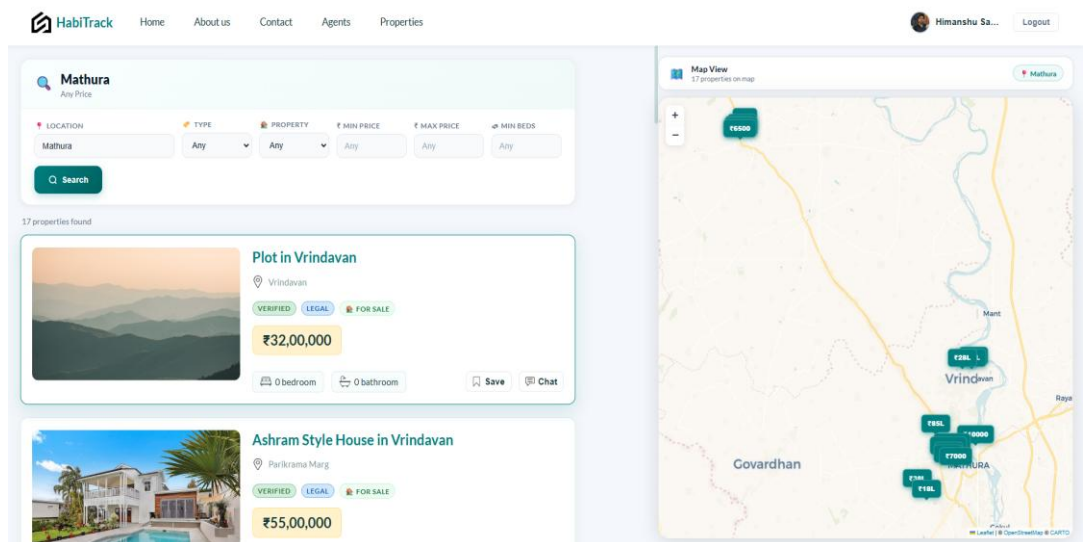


Figure 5.3.3 Property Listing Page

4. Search Property Listing Page

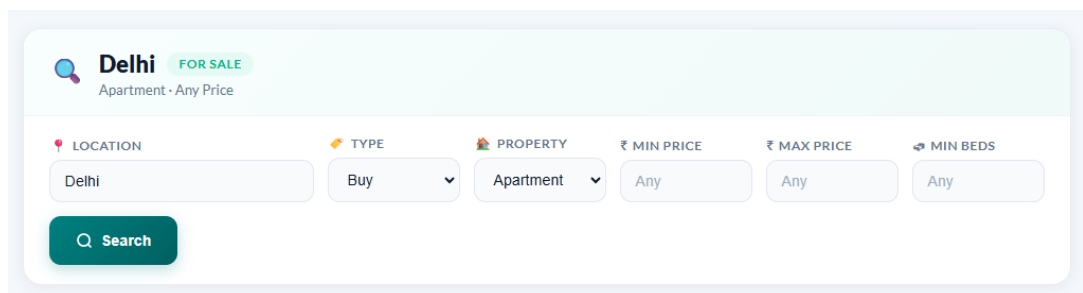


Figure 5.3.4 Search Property Listing Page

5. Contact Page

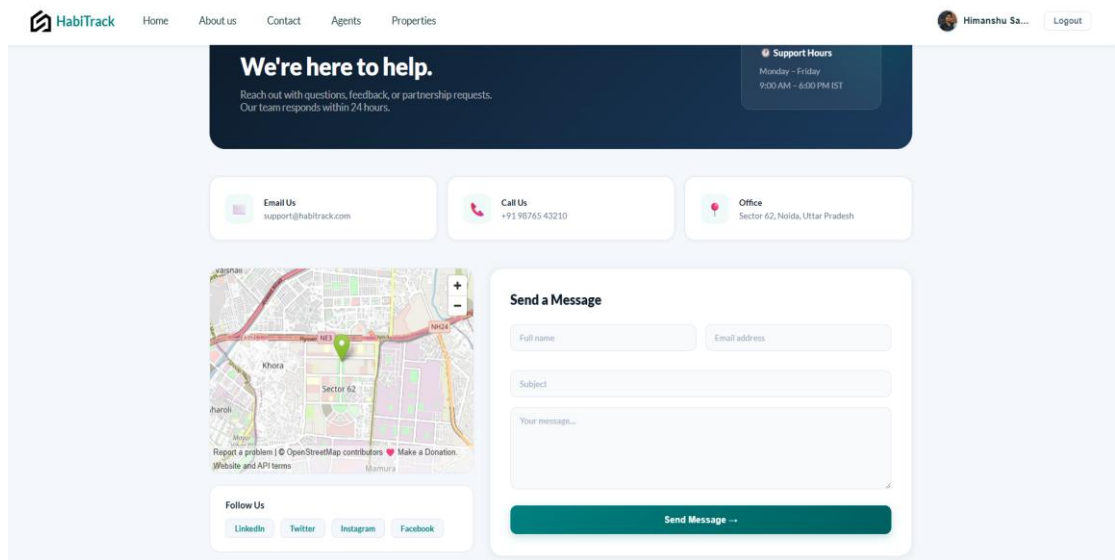


Figure 5.3.5 Contact Page

6. Owner Profile & Chat Page

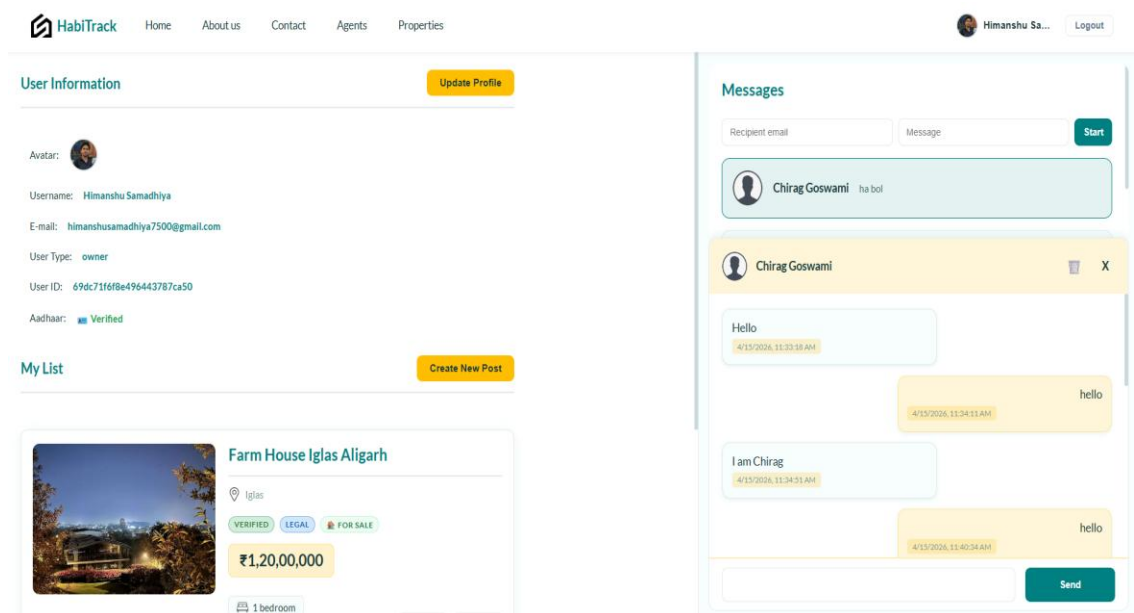


Figure 5.3.6 Owner Profile & Chat Page

7. Property Buy/rent Payment page

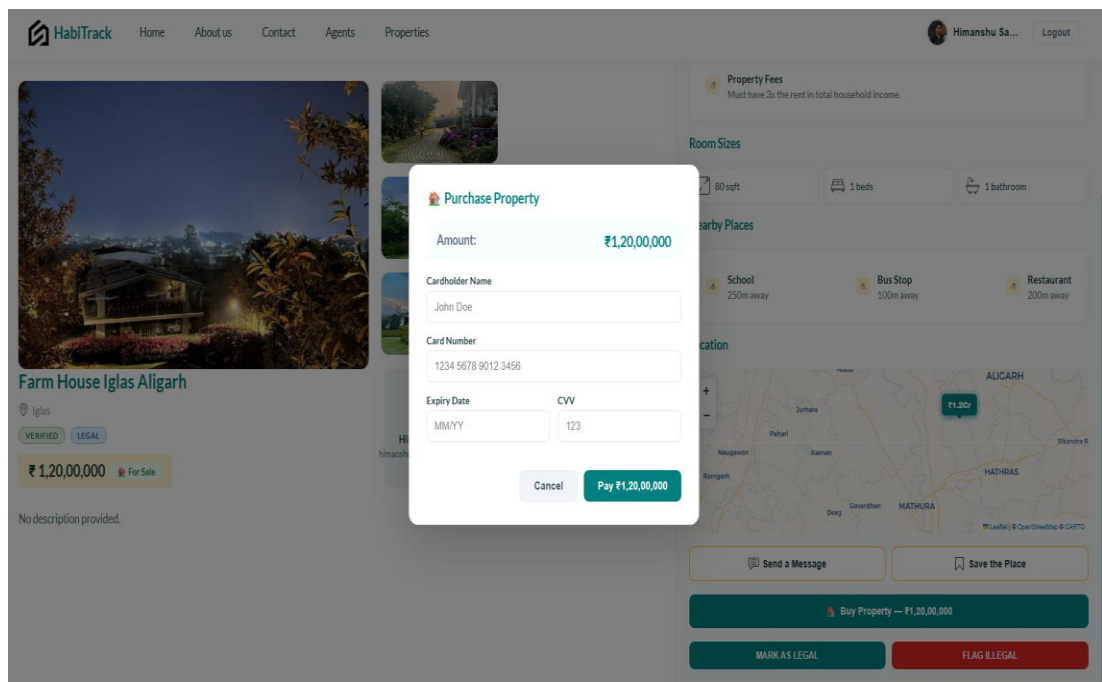


Figure 5.3.7 Property Buy/rent Payment Page

8. Admin Dashboard

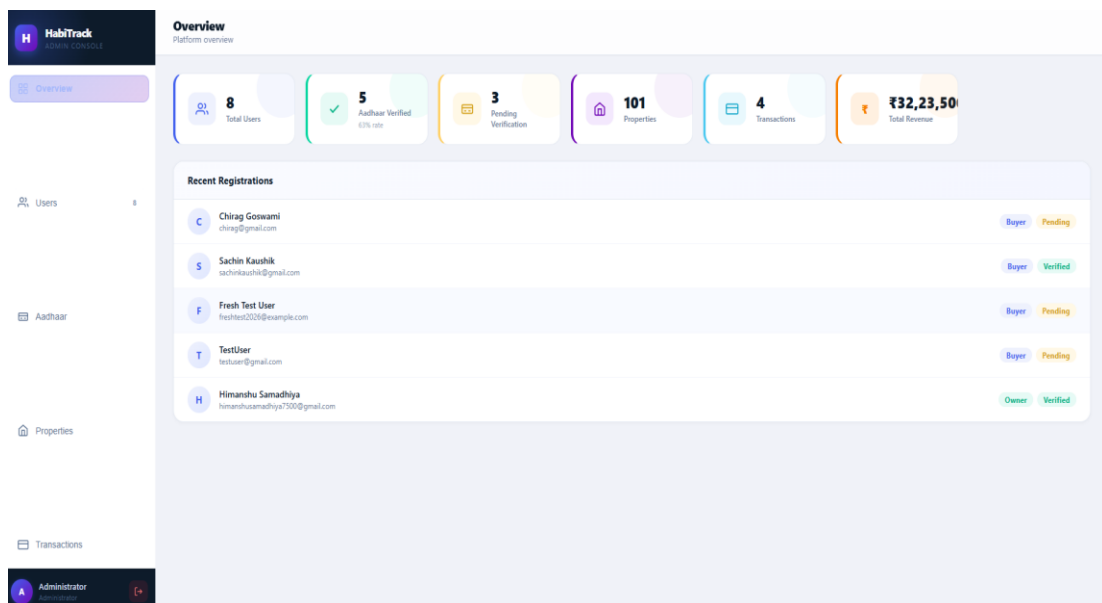
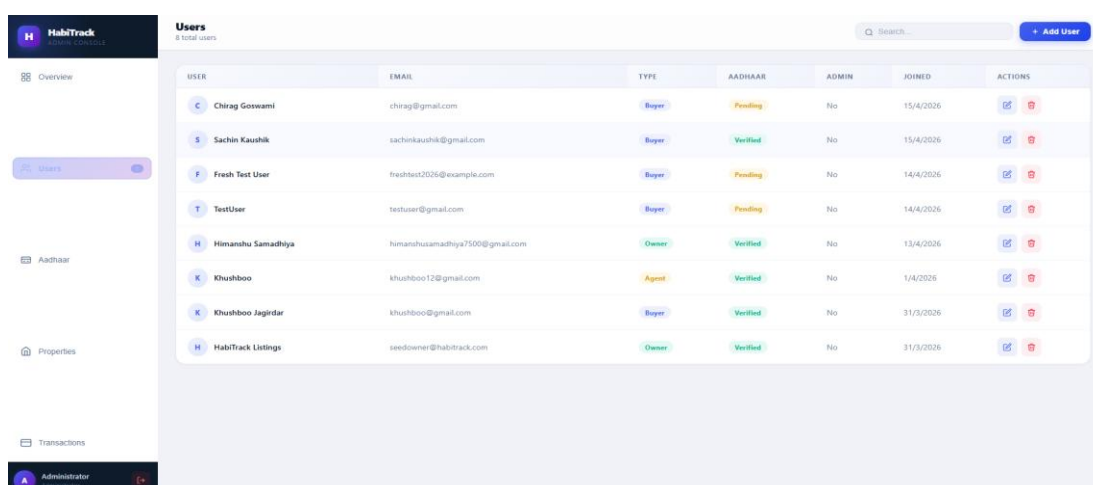


Figure 5.3.8 Admin Dashboard

9. User in Admin Dashboard



USER	EMAIL	TYPE	AADHAAR	ADMIN	JOINED	ACTIONS
C Chirag Goswami	chirag@gmail.com	Buyer	Pending	No	15/4/2026	[Edit] [Delete]
S Sachin Kaushik	sachinkaushik@gmail.com	Buyer	Verified	No	15/4/2026	[Edit] [Delete]
F Fresh Test User	freshtest2026@example.com	Buyer	Pending	No	14/4/2026	[Edit] [Delete]
T TestUser	testuser@gmail.com	Buyer	Pending	No	14/4/2026	[Edit] [Delete]
H Himanshu Samadhiya	himanshusamadhiya7500@gmail.com	Owner	Verified	No	13/4/2026	[Edit] [Delete]
K Khushboo	khushboo12@gmail.com	Agent	Verified	No	1/4/2026	[Edit] [Delete]
K Khushboo Jagirdar	khushboo@gmail.com	Buyer	Verified	No	31/3/2026	[Edit] [Delete]
H Habitrack Listings	seedowner@habitrack.com	Owner	Verified	No	31/3/2026	[Edit] [Delete]

Figure 5.3.9 User in Admin Dashboard

10. Payment Interface

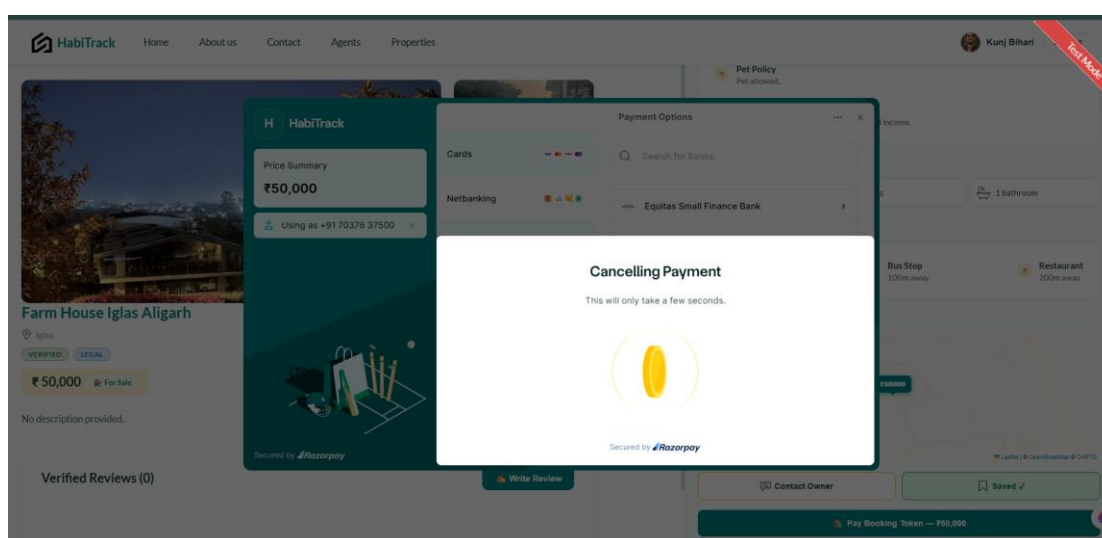


Figure 5.3.10 Payment Interface

Chapter 6

Conclusion

The HabiTrack system successfully fulfils the objective of providing a centralized, efficient, and user-friendly platform for property search, management, and transactions. By integrating property listings, map-based search, chat, transactions, and Aadhaar verification, the system improves the overall real estate experience and reduces dependency on multiple external platforms.

The Buyer/Renter Module provides users with a simple environment to search, filter, save, and purchase or rent properties. The Property Owner/Agent Module allows owners and agents to manage listings, update property details, and interact with buyers. The Admin Module ensures complete administrative control, maintaining security, proper user management, and smooth system operation.

Overall, the system satisfies all major functional requirements such as authentication, property management, transactions, messaging, and analytics. It also meets non-functional requirements like reliability, usability, scalability, maintainability, and security. The project demonstrates how modern web technologies can simplify and improve the real estate process.

Chapter 7

Summary

This project aimed to create a comprehensive real estate platform to assist users in searching, buying, renting, and managing properties efficiently. The system brings together important features such as property listings, map integration, chat, transactions, Aadhaar verification, and analytics into a single platform.

The report began with an introduction to the problem statement, objectives, and motivation behind the system. Chapter 2 discussed the requirement analysis, module descriptions, and UML/DFD diagrams that shaped the system architecture. Chapter 3 explained the design methodology and system diagrams, while Chapter 4 detailed implementation aspects such as technologies used, backend and frontend development, and module integration. In Chapter 5, different testing techniques were applied to ensure system stability and correctness, and the results showed successful execution of all major functionalities.

The system provides value to buyers, property owners, agents, and administrators. Buyers benefit from easy property search and communication features, owners and agents can efficiently manage listings, and administrators maintain control over users, properties, and transactions. The project demonstrates a reliable, scalable, and effective solution for modern real estate management, with scope for future enhancements.

Appendices

Appendix 1 – Project Summary

HabiTrack is a full-stack real estate web application designed to simplify property searching, buying, renting, and management. It integrates features such as property listing, map-based search, chat communication, Aadhaar verification, and transaction handling. The system ensures a centralized, user-friendly, and efficient platform for buyers, property owners, agents, and administrators.

Appendix 2 – Tools & Technologies

- **Frontend/Backend:** MERN Stack (MongoDB, Express.js, React, Node.js)
- **Database:** MongoDB Atlas
- **Authentication:** JWT, bcrypt, Aadhaar OTP Verification
- **Deployment:** Localhost / Cloud (vercel, Render)
- **Maps Integration:** Leaflet.js, OpenStreetMap

Appendix 3 – Hardware Requirements

- Laptop with Core i5/i7 processor
- Minimum 4-8 GB RAM
- Stable internet connection

Appendix 4 – Software Requirements

- Windows 10/11
- Code Editor (VS Code)
- Web Browser (Chrome/Edge)
- Node.js (LTS version)

Appendix 5 – Major Functional Modules

- User Registration & Login
- Buyer/Renter Module (Search, Save, Buy/Rent)
- Property Owner/Agent Module (Add & Manage Listings)
- Chat & Messaging System
- Transaction Management
- Admin Dashboard & Analytics

Appendix 6 – Gantt Chart (Summary)

- Requirements: Sep 2025
- Research: Sep–Oct
- Design: Oct–Nov
- Development: Nov–Jan
- Testing: Jan–Feb
- Deployment: Mar

Appendix 7 – Use Case (Brief)

- UC1: User logs in/registers
- UC2: User searches properties with filters
- UC3: User views property details and location
- UC4: User buys/rents property
- UC5: User communicates with owner/agent
- UC6: Admin manages users and properties

Appendix 8 – System Flow (Summary)

User/Admin → Authentication → Property Search/Listings → Interaction (Chat/Save)
→ Transaction → Data Storage → Result Display

Appendix 9 – Interface List

- Home Page
- Login / Signup Page
- Property Listing Page
- Property Detail Page
- User Profile Dashboard
- Chat Interface
- Admin Dashboard

Appendix 10 – Future Scope (Brief)

- Mobile application development
- Advanced recommendation system using AI
- Integration with real payment gateways
- Enhanced security and verification features
- Expansion for large-scale real estate platforms

References

- [1] K. H. Rosen, *Introduction to Web Development*, Pearson Education, 2018.
- [2] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed., Pearson, 2017.
- [3] MongoDB Inc., “MongoDB Documentation,” [Online]. Available: <https://www.mongodb.com>
- [4] Express.js, “Express – Node.js Web Application Framework,” [Online]. Available: <https://expressjs.com>
- [5] React Team, “React Documentation,” [Online]. Available: <https://react.dev>
- [6] Node.js Foundation, “Node.js Documentation,” [Online]. Available: <https://nodejs.org>
- [7] OpenStreetMap Foundation, “OpenStreetMap,” [Online]. Available: <https://www.openstreetmap.org>
- [8] Leaflet.js, “Leaflet – JavaScript Library for Interactive Maps,” [Online]. Available: <https://leafletjs.com>
- [9] OWASP Foundation, “OWASP Top Ten Web Application Security Risks,” [Online]. Available: <https://owasp.org>
- [10] Razorpay, “Razorpay Payment Gateway Documentation,” [Online]. Available: <https://razorpay.com/docs>
- [11] M. Mikowski and J. Powell, *Single Page Web Applications: JavaScript End-to-End*, Manning Publications, 2013.



